The first part specifies that system tests should use Rack::Test, which is Capybara's internal Ruby driver, the one without RSpec. This setup is specific to Rails 5.1 and RSpec 3.7 and later. The second part sets you up to have capybara-screenshot available in your specs.

Capybara and Its Own language

At this point you're forced to confront another library syntax decision. Capybara defines its validations mostly as things that look like RSpec matchers. It also defines its own optional integration-test syntax, similar to RSpec but with terms like `feature` and `scenario` to drive home the integration-testing point.

All of this adds up to a lot of different ways to express the same test. In the interest of minimizing the amount of time I spend talking about syntax, which is the least interesting thing I could be talking about, I'll stick to the same basic RSpec syntax you've been using in most of the book. Keep in mind that the other forms exist; that way when your coworker shows you her Capybara tests, you know how to read them.

Why are there so many kinds of tests? There are basically three reasons:

- *Functionality*. Because the different kinds of tests cover different parts of the process, they are useful for different kinds of code.
- *Taste*. Some people like writing unit tests for mailers, helpers, and the like, while others prefer to create coverage with end-to-end tests. If you're David Heinemeier Hansson, your taste preferences can lead to significant features of controller tests being deprecated.
- *Genuine tool improvement over time*. The Capybara system tests are easier to write and understand than the original Rails integration tests.

Later in this chapter you'll learn about RSpec's system and feature, and take a look at Cucumber, a different tool for running integration tests.

Using Feature Tests to Build a Feature

When last you left your application, you allowed for the creation of new projects. Let's follow up on that and add a sequence where you can see a page for an existing project and add a task to it. To give this a little bit of back-end logic to play with, let's set up a situation where the tasks are ordered and then move one task above the other task.